other middleware in the pipeline and also ensures that you don’t try to modify the headers after the response has started streaming to the browser.

Listing 31.4 Adding headers to a response with the `Use` extension

```
app.Use(async (HttpContext context, Func<Task> next) =>                ❶
{
    context.Response.OnStarting(() =>                                  ❷
    {
        context.Response.Headers["X-Content-Type-Options"] = "nosniff"; ❸
        return Task.CompletedTask;                                     ❹
    });
    await next();                                                      ❺
}

app.UseStaticFiles();                                                  ❻
app.UseRouting();                                                      ❻

app.MapRazorPages                                                      ❻
```

❶ Adds the middleware at the start of the pipeline

❷ Sets a function that runs before the response is sent to the browser

❸ Adds the header to the response for added protection against XSS attacks

❹ The function passed to OnStarting must return a Task.

❺ Invokes the rest of the middleware pipeline

❻ No matter what response is generated, it’ll have the security header added.

Simple cross-cutting middleware like the security header example is common, but it can quickly clutter your Program.cs configuration and make it difficult to understand the pipeline at a glance. Instead, it’s common to encapsulate your middleware in a class that’s functionally equivalent to the `Use` extension but that can be easily tested and reused.

31.1.4 Building a custom middleware component